

2026 Logic System Lab

邏輯系統實驗

Lab6 Report

	姓名	學號
組員 1	何家睿	E24146644
組員 2	張家榕	F84131251

組別：01

(注意:時間常數皆有 for implement 跟 for simulation, 視使用需求自行註解)

[Summary]

(我很抱歉忘記蓋衛生紙，已經來不急了)

	Finished / Unfinished	YouTube Link
P1	Finish	https://youtube.com/shorts/GiRkt9NhcnM?si=NETQvlpBk2UQGdvy
P2	Finish	https://youtube.com/shorts/TOKDM1nC1kE?si=aZhcjY6lbfqp_otW

[Practice 1]

Q1: Please explain your overall design concept

Q1-1: 在同一周期內閃爍不同頻次數，因為週期很短，對人眼來說就能達到混色，即 PWM 的原理。

Q1-2(fsm 與系統設計):

這是一個具有七種顏色變化的呼吸燈。核心邏輯分為三個層次：時間時序控制、顏色狀態切換、以及 PWM 呼吸效果產生。

A. 時間時序控制

為了達成 1 秒換色與 0.5 秒漸亮漸暗，系統設計了三種計數機制：

- 總計時器 (timer)：以 125 MHz 為基準，計數 1 秒 (CYCLE_10) 作為顏色切換的週期。
- 多的：步進計數器 (step_timer)：將 0.5 秒切割為 100 階 (每階約 5 ms)，每達到 CYCLE_STEP 時觸發一次亮度增減。**避免 % (取餘數) 運算**
- 相位計數器 (phase_counter)：PWM 計數器 (0-199)，決定 LED 的閃爍頻率以達到調變光的顏色。

B. 有限狀態機 (FSM)

- 狀態定義：包含 RED, ORANGE, YELLOW, GREEN, BLUE, PURPLE, WHITE。
- 轉移邏輯：每當 passed_1s 訊號觸發 (計時滿 1 秒)，狀態機即跳轉至下一個顏色。
- 輸出邏輯：根據目前狀態，設定 threshold_r/g/b 的基礎比例，決定該顏色由哪些原色組成。
- rst 回 RED

C. 呼吸效果與 PWM 輸出邏輯

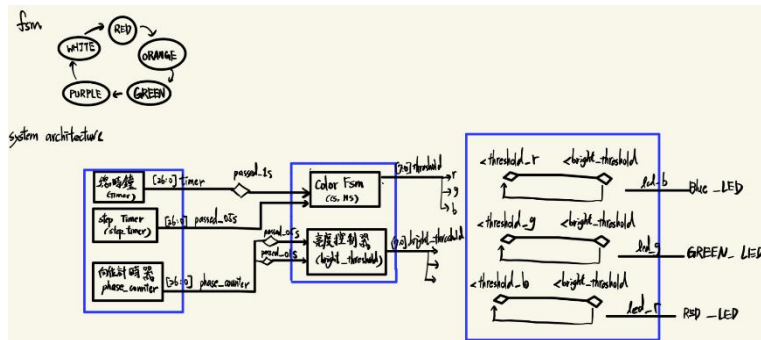
這是本實驗最關鍵的部分，最終輸出由兩個條件共同決定：

- 呼吸亮度 (bright_threshold)：
 - 前 0.5 秒 (!passed_05s)：亮度閾值遞增。
 - 後 0.5 秒：亮度閾值遞減。

```
assign led_x = (phase_counter < threshold_x) && (phase_counter < bright_threshold);
```

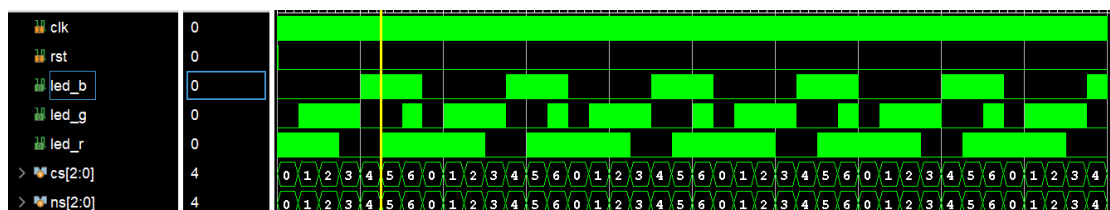
這行邏輯是 LED 「顏色比例」與「呼吸亮度」的雙重控制。只有當相位計數器同時小於這兩個閾值時，LED 才會點亮，進而達成 **漸亮漸暗** 跟 **彩色** 的呼吸效果。

Q1-3(流程圖與架構圖):

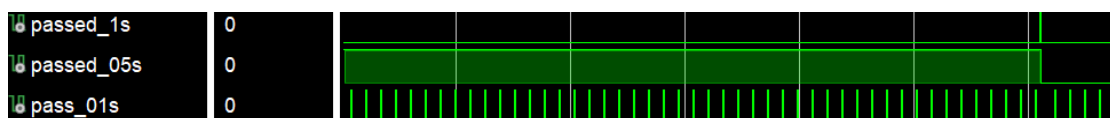


Q2: Screenshot the waveform and explain how to determine if it is functioning correctly

- (一)led_r/g/b 已設定比例被拉高，實現 PWM 混色。
- (二)cs 從 0 到 6，再從 6 到零。變色狀態機功能正確。
- (三)最左邊 rst 拉高，全部歸零。



- #### (四)因時鐘而觸發的幾個時間旗標正確運行



- (五) `bright_threshold` 代表此刻亮度，確實「先亮在暗」呈現單顏色的呼吸效果。



[Practice 2]

Q1: Please explain your overall design concept

這是一個可透過按鈕調整切換速度的 RGB 呼吸燈。延續 pracice1，加入幾個不同的計時限制。和去彈跳與長按偵測功能

Q1-1 top module

- A. fsm: 換色邏輯同 practice_1，只是時間間隔會因當前 mode 有所改變。
- B. 時間限制參數化

不同於第一題使用固定常數，這次定了四種速度模式（2.0s, 1.0s, 0.5s, 0.25s）。

- **邏輯**：透過 `always @(*)` 組合電路，根據 `mode` 值動態設定 `color_limit` 與 `step_limit`。
- **比例維持**：為了保證在不同速度下呼吸效果依然平滑，`step_limit` 是依據當前時間限制除出來的 `color_limit / 200`，確保每個呼吸週期都能走完整的 100 階亮度增減。`dim_limit = color_limit >> 1;`

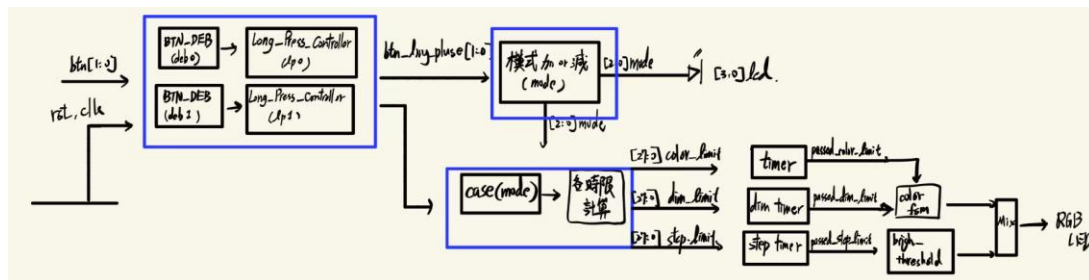
B. 輸入處理鍵

1. `BTN_DEB`（去彈跳）：濾除按鈕按下瞬間的機械雜訊（Spike）。
2. `Long_Press_Controller`（長按控制）：偵測按鈕按下的時長，判斷為有效操作後才送出單脈衝訊號（`out_pulse`），避免誤觸或連續觸發。
3. 模式控制：`btn[0]` 與 `btn[1]` 分別控制速度的「加」與「減」，並透過 `if(mode < 3'd3)` 邏輯設定邊界保護，防止數值溢位。

C. 模式燈

除了 RGB LED 的速度變化外，設計中額外使用了 4-bit 的單色 LED（`output [3:0] led`）作為速度指示燈。

Q1-3 架構圖：



Q1-4 長按連發控制邏輯

當使用者短按時，系統會立即反應；若持續按住，則會自動進入「連發模式」。

A. 運作階段說明

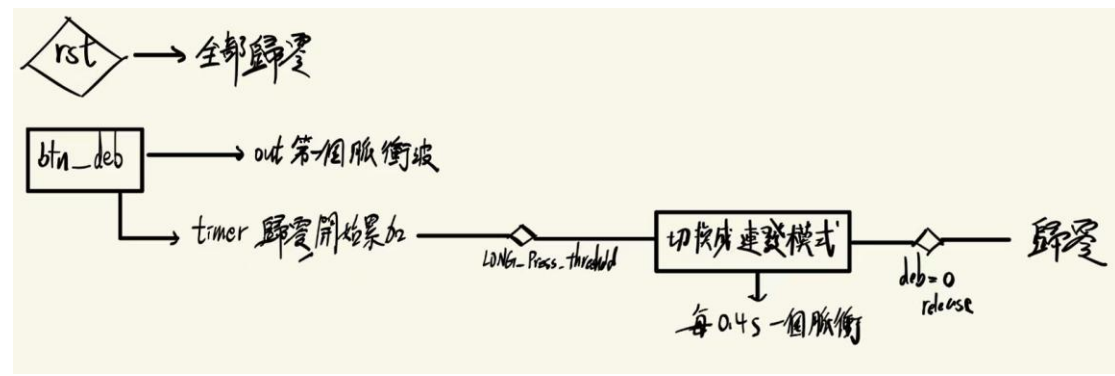
該模組根據按鈕按下的時長，將行為分為三個關鍵點：

1. **瞬發階段**：
 - 當偵測到按鈕 `btn_deb` 被按下的第一個時脈週期（`timer == 0`），立即將 `out_pulse` 拉高。這確保了「點按」操作的零延遲感。
2. **等待閾值**：
 - 持續按住時，計數器開始累加。系統會等待 `LONG_PRESS_THRESHOLD`（本設計設定為 1 秒，程式碼註解為 0.5s，需留意數值與註解的一致性）。在此期間不發出任何脈波，用以區分「點按」與「長按」。
3. **連發模式**：
 - 一旦計數器超過閾值，`long_press_mode` 轉為 1。此時進入週期性觸發狀態，每隔 `REPEAT_INTERVAL`（0.4 秒）就會自動送出一個脈波，達成自動連加/連減速度的效果。
 -

B. 狀態重置機制

一旦 btn_deb 為低位準（使用者放開手指），無論當前處於哪一個階段，計數器與 long_press_mode 都會立即歸零。下一次按下時，邏輯會從「瞬發階段」重新開始。

架構圖：

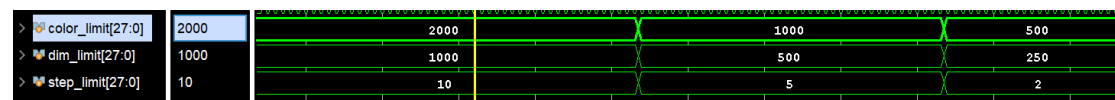


Q3: Screenshot the waveform and explain how to determine if it is functioning correctly

(一)觀察 mode 的變化，可以發現當 mode 增加 pass_step(漸亮漸暗)的頻率更高。可驗證 practice2 不同速度模式的切換邏輯。

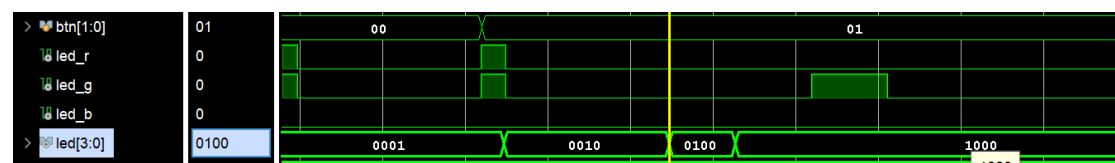


(二)觀察同樣的各時鐘的時間限制都有被 mode 正確觸發。



(三)當前模式 led 指示燈有正確點亮。

(四)觀察 btn 跟 led 有實現”長按進入連發模式”。



[Linter]

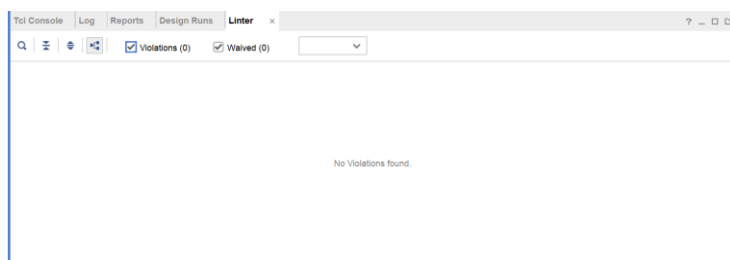
(運氣極好)

	# warnings	# errors
P1	0	0
P2	0	0

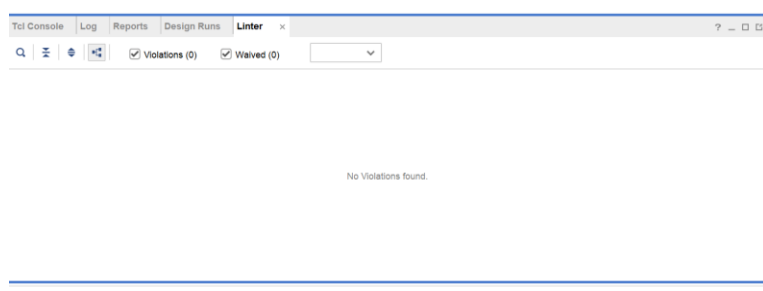
Provide screenshots of the remaining Linter warnings and errors below,

including root cause analysis and why they are currently unresolvable

P1



P2



[心得]

組員一 何家睿 E24146644

在這次為期兩週的 Lab6 中，我們小組致力於重新定義合作。我們選擇從兩人共用螢幕、一起討論的摸索階段開始，希望能杜絕以往用 AI 趕工以及組內隱形競爭的狀態。實驗證明，這樣的調整帶來了良好的互動效果。

雖然，在緊迫的時間限制下，這種步調初期的確降低了效率，但它對我的意義卻是深遠的。『模仿（參考範例）』到『類推（動手實作）』的過程，遠比過去趕快叫 ai 生踏實(tb 還是有請 ai 幫忙)。儘管遺憾自己沒有及早養成這種主動內化的學習習慣，但已經跨出了很大的一步。希望會越來越好。

組員二 張家榕 F84131251

在 Lab6 中，我與組員一起合作完成了 Practicel，這是我們第一次一起在課堂上完成，以往都是各自思考解決，很意外的是跑 synthesis 的時候沒有跳出 error，我覺得就這次的合作而言還是很好的體驗，雖然我不見得幫上了什麼忙，但合作完成促進了組內溝通與理解，比自己一個人想還要有趣很多。